HealthNexus
Nira and the RAG pipeline

How it answers only related questions, the 29 queries, the pipeline, how to defend the name, the files, the stack and the speed

For reading and for defence: every query table is generated from the live catalog; every timing was measured on this machine.
Generated 27 October 2026 from the live code. Every path:line link opens VS Code at that line.
Open any line by hand: Ctrl+P, paste the path and line, press Enter.

Contents

1. The big picture: what Nira's RAG is 3

2. How it answers only the correct, related question 4

3. The 29 queries: what they are, why, and how they operate 6

4. The pipeline, end to end 10

5. Defending the name 'RAG' 12

6. Folder structure: what each file does 15

7. Tech stack and response time 17

8. Defence Q&A and known limits 20

1 The big picture: what Nira's RAG is

A question is answered in four moves: pick a vetted query, run it safely, word the answer from its rows, and check the numbers.

29	3	6	4 ms	~5 to 6 s
vetted SQL queries	roles, each with its own list	files in the assistant package	to answer without Gemini (median)	to answer with Gemini

Question → Safety check → Choose a query → Run it, read-only → Gemini words the answer from the rows → Check every number → Show with its sources

Step	What happens	Who does it	File
1 Safety	Emergency words get 'call 108'; diagnosis and medicine questions are declined; greetings get help	Fixed rules	app/services/assistant/router.py:24
2 Plan	Pick **which query** answers it and its parameters	Gemini (function calling), or the keyword router if Gemini is unavailable	app/services/assistant/__init__.py:184
3 Retrieve	Run that query for **this user only**, read-only, with a time limit	PostgreSQL	app/services/assistant/__init__.py:141
4 Generate	Write a short answer using only the returned rows	Gemini, or a template	app/services/assistant/llm.py:133
5 Verify	Every number in the answer must appear in the rows	Code	app/services/assistant/grounding.py:56
6 Record	Store the answer with its sources so it can be shown and audited	Code	app/services/assistant/__init__.py:333

> **What makes it RAG:** the model never sees the database and never writes SQL. It chooses from a fixed menu, the server fetches the data, and the answer is generated **from that retrieved data** and checked against it. Chapter 5 gives the full defence.

Who can ask what

Role	Queries open	Sees
Citizen	11	Only their own rows (`user_id = :me`)
PHC worker	11	Their centre's catchment: citizens who have not withdrawn worker access
Admin	10	Aggregates and account metadata for their district; **never anyone's readings**

2 How it answers only the correct, related question

Nothing is left to the model's judgement. Seven layers, each in a known file, stand between a question and an answer.

#	Layer	File responsible	What it guarantees	Code
1	**Emergency and safety rules**	router.py	Chest pain, stroke, 'can't breathe' and similar get 'call 108' immediately; 'what medicine should I take', 'do I have dengue' are declined for citizens	app/services/assistant/router.py:19
2	**Role whitelist**	catalog.py	A role can only be offered its own queries (`tools_for(role)`); a citizen can never be offered `flagged_citizens`	app/services/assistant/catalog.py:740
3	**A fixed menu, not free SQL**	catalog.py	The model or router can only choose a query name and fill its declared parameters. There is no way to ask for a different query	app/services/assistant/catalog.py:81
4	**Planner instructions**	llm.py	Gemini is told to call a function only if one fits, else reply with a short sentence; temperature 0 so the choice is repeatable	app/services/assistant/llm.py:26
5	**Parameter cleaning**	catalog.py	Every argument (days, limit, severity, name) is coerced and clamped; anything odd falls back to the default	app/services/assistant/catalog.py:64
6	**No match means no answer**	__init__.py	If no query fits, the reply is 'I can only answer questions about HealthNexus records' with examples, and it is stored as an out-of-scope refusal	app/services/assistant/__init__.py:238
7	**Answer prompt and number check**	llm.py, grounding.py	Gemini must use only the rows and ignore instructions inside them; any number not in the rows discards the answer	app/services/assistant/llm.py:33

Live behaviour (computed from the real rules)

The table runs each question through the actual safety rules and the rule-based planner. Gemini's planner is more flexible but is held to the same menu.

Role	Question	What happens	Decided by
Citizen	What was my blood pressure yesterday?	Runs the query `my_latest_reading` (match score 1.5).	catalog + router
Citizen	Hello	Greeting / help text listing example questions.	router.py
Citizen	I have chest pain	**Emergency reply**: 'call 108 now'. No query and no model call.	router.py
Citizen	Should I take paracetamol for my fever?	**Declined**: 'I can't diagnose or advise on medicines'.	router.py
Citizen	What is the capital of France?	**Refused**: 'I can only answer questions about HealthNexus records', plus examples.	catalog + router
Citizen	Show me Priya's readings	**Refused**: 'I can only answer questions about HealthNexus records', plus examples.	catalog + router
Citizen	How many citizens are flagged in my catchment?	Runs the query `my_flags` (match score 3).	catalog + router
Worker	Who has critical flags right now?	Runs the query `flagged_citizens` (match score 7).	catalog + router
Worker	Show me the readings of HN-1A2B3C4D	Runs the query `citizen_summary` (match score 5).	catalog + router
Worker	What is 2 + 2?	**Refused**: 'I can only answer questions about HealthNexus records', plus examples.	catalog + router

Role	Question	What happens	Decided by
Admin	Give me a platform overview	Runs the query `platform_overview` (match score 3).	catalog + router
Admin	Show me anyone's heart rate readings	**Refused**: 'I can only answer questions about HealthNexus records', plus examples.	catalog + router
Admin	Delete all users	**Refused**: 'I can only answer questions about HealthNexus records', plus examples.	catalog + router

> **Be ready to admit this limit:** the keyword fallback is crude. 'How many citizens are flagged in my catchment?' asked by a *citizen* matches the word 'flagged' and answers about **their own** flags. It is wrong-but-safe: the query is scoped to that citizen, so nothing private leaks. With Gemini planning the question is understood semantically; on the rules path a stricter minimum match score would refuse it.

Which file does what, in one view

File	Role
app/services/assistant/router.py:17	Safety and intent rules (emergency, medical advice, greetings, help) and the rule-based planner `plan`.
app/services/assistant/catalog.py:152	The 29 queries, their parameters, role lists and keyword patterns.
app/services/assistant/llm.py:104	Gemini: choose up to three queries; word the answer.
app/services/assistant/__init__.py:217	The conductor that applies every layer in order.
app/services/assistant/grounding.py:56	The number check.
app/services/assistant/compose.py:68	Template answers used when Gemini fails or its answer is rejected.

Questions you may be asked

Q. What stops the model answering something unrelated, like the weather?

Two things. The planner can only pick from the role's 29-or-fewer queries and is told to reply briefly if none fits, so no query runs. And even if a query were chosen, the answer prompt limits the model to the retrieved rows.

Q. What stops it reading another user's data?

The SQL is fixed and written by us, not the model. `:me`, `:phc_id` and `:district` are bound on the server from the signed-in user, never from the question. A model that tried to ask for 'Priya's readings' as a citizen has no query for it.

3 The 29 queries: what they are, why, and how they operate

Each query is one vetted SQL statement with declared parameters. They were chosen to cover what each role already does on screen.

3.1 Why these 29

- **They mirror the app's own screens.** Each query answers a question a person already asks of a page (latest reading, flagged list, schedule, pending applications). The assistant is a faster way in, not a new data source.
- **They come from the suggested questions** shown on Nira's start screen for each role, plus the newer features (examinations, advisories, outbreak signals, smartwatch sources).
- **Coverage versus safety:** every query is reviewed once for scope. A new question is one new query, not a new risk. The cost is that only questions a query covers are answered; the rest are politely refused.
- **Privacy is built into the SQL.** Worker queries use the same catchment rule as the screens; area-level queries include only opted-in citizens and fold groups under 3; admin queries return totals, never readings.

Role	Count	Queries
Citizen only	10	`my_latest_reading`, `my_vital_stats`, `my_activity`, `my_flags`, `my_visits`, `my_health_centre`, `my_profile`, `my_examinations`, `my_readings_by_source`, `my_advisories`
Worker only	8	`catchment_overview`, `flagged_citizens`, `citizen_summary`, `silent_citizens`, `my_schedule`, `locality_breakdown`, `catchment_trend`, `symptom_counts`
Admin only	8	`platform_overview`, `pending_applications`, `facility_load`, `unlinked_workers`, `concerns_summary`, `signups`, `district_activity`, `assistant_usage`
Shared	3	`my_alerts` (Worker and Citizen), `advisories_overview` (Admin and Worker), `outbreak_signals` (Admin and Worker)

3.2 Citizen queries (own rows only)

#	Query	Title	Why it exists	Tables read	Parameters
1	`my_latest_reading`	Your latest reading	The most common question: 'is my last reading normal?' Mirrors the dashboard's latest values.	health_records, processed_data	none
2	`my_vital_stats`	Your vitals over time	Average, lowest and highest per vital over a period; mirrors the dashboard trend and History.	health_records	days
3	`my_activity`	How often you have logged	How often they log; mirrors the dashboard's weekly count.	health_records, processed_data	days
4	`my_flags`	Your flagged readings	Which readings were flagged and why; mirrors History's 'flagged only' filter.	health_records, processed_data	days
5	`my_alerts`	Your notifications	The notification list; mirrors the Alerts page (a worker's Notifications too).	alerts	limit
6	`my_visits`	Your visits	Upcoming and recent visits; mirrors the Visits page.	scheduled_visits, users	none
7	`my_health_centre`	Your health centre	Which centre looks after them and how many workers it has; mirrors Settings → Your health centre.	users, valid_phc_facilities	none
8	`my_profile`	Your health profile	What the app holds about them; mirrors the Profile page.	user_profiles	none

#	Query	Title	Why it exists	Tables read	Parameters
9	`my_examinations`	Your examinations	Examinations a health worker recorded: when, where, by whom.	health_records, users	days
10	`my_readings_by_source`	Where your readings came from	How many readings came from the watch, an examination or typing, and the latest of each.	health_records	days
11	`my_advisories`	Advisories for you	Advisories currently running for where they live; matches the Alerts page.	advisories, users	none

3.3 Worker queries (their catchment)

#	Query	Title	Why it exists	Tables read	Parameters
1	`my_alerts`	Your notifications	The notification list; mirrors the Alerts page (a worker's Notifications too).	alerts	limit
2	`catchment_overview`	Your catchment at a glance	The dashboard's counts as one answer: citizens, flagged, readings, visits.	consent_records, health_records, processed_data, users	days
3	`flagged_citizens`	Citizens with open flags	The worklist: who has open flags, worst first; mirrors the Alerts page.	consent_records, health_records, processed_data, users	severity, limit
4	`citizen_summary`	Citizen lookup	One citizen's status, by name or HN reference; mirrors the citizen record.	consent_records, health_records, processed_data, scheduled_visits, users	name, days
5	`silent_citizens`	Citizens who have gone quiet	Who has not logged anything for N days; mirrors the Citizens page 'gone quiet'.	consent_records, health_records, users	days, limit
6	`my_schedule`	Your visits	The worker's visits in the next days; mirrors the Schedule page.	scheduled_visits, users	days, limit
7	`locality_breakdown`	Flags by locality	Which localities have the most flags (only opted-in citizens, never under 3); mirrors the risk map.	consent_records, processed_data, users	none
8	`catchment_trend`	Readings and flags per day	Readings and flags per week; mirrors the dashboard KPIs.	consent_records, health_records, processed_data, users	days
9	`symptom_counts`	Most reported symptoms	Which symptoms are being reported (opted-in citizens); supports outbreak watching.	consent_records, health_records, users	days
10	`advisories_overview`	Advisories published	Advisories published in a window, with status and feedback; mirrors the Advisories history.	advisories	days, limit
11	`outbreak_signals`	Outbreak signals	Outbreak clusters flagged in a window; mirrors the Outbreaks page.	community_data	days

3.4 Admin queries (aggregates for their district)

#	Query	Title	Why it exists	Tables read	Parameters
1	`platform_overview`	Platform overview	Accounts, readings, open flags, applications and concerns for the admin's district.	concerns, health_records, phc_applications, processed_data, users	days
2	`pending_applications`	PHC applications waiting	PHC applications waiting; mirrors the Applications page.	phc_applications, users	limit
3	`facility_load`	Health centres by load	Citizens and open flags per health centre; shows imbalance.	processed_data, users, valid_phc_facilities	limit

#	Query	Title	Why it exists	Tables read	Parameters
4	`unlinked_workers`	Workers with no health centre	Approved workers with no centre, who therefore see nobody.	users	none
5	`concerns_summary`	Support concerns	Complaints by status and category; mirrors the Complaints page.	concerns, users	none
6	`signups`	New accounts	New accounts by role over a period.	users	days
7	`district_activity`	Activity by district	Per district activity (opted-in citizens, groups of 3 or more).	consent_records, health_records, processed_data, users	days
8	`assistant_usage`	Assistant usage	How Nira itself is being used: questions, unanswered, latency.	chat_messages, users	days
9	`advisories_overview`	Advisories published	Advisories published in a window, with status and feedback; mirrors the Advisories history.	advisories	days, limit
10	`outbreak_signals`	Outbreak signals	Outbreak clusters flagged in a window; mirrors the Outbreaks page.	community_data	days

3.5 How one query operates

Here is the real definition of `my_vital_stats`, the answer to 'what were my average vitals over the last 30 days?':

```
SELECT v.key AS vital, count(*) AS readings,
       round(avg(v.value::numeric), 1) AS average,
       min(v.value::numeric) AS lowest, max(v.value::numeric) AS highest
FROM health_records hr
CROSS JOIN LATERAL json_each_text(hr.vitals) AS v(key, value)
WHERE hr.user_id = :me AND hr.timestamp >= now() - make_interval(days => :days)
GROUP BY v.key
ORDER BY v.key
```

Part	What it is
`:me`	A **server-side bind**: the signed-in user's id. It is not a parameter the model can set.
`days`	A declared parameter: integer, 1 to 365, default 30. The model's value is cleaned by `Param.clean`.
The SQL text	Written by us, reviewed once. The model can never edit it.
Keyword patterns	Weighted regular expressions used only by the rule-based planner when Gemini is unavailable.
`description`	The sentence Gemini reads when choosing among queries (function calling).

Safeguard on every query	Value	Code
Read-only transaction	`SET TRANSACTION READ ONLY`	app/services/assistant/__init__.py:145
Time limit	5 seconds (`statement_timeout`)	app/services/assistant/__init__.py:45
Rows read from the database	at most 200	app/services/assistant/__init__.py:43
Rows stored as the visible source	at most 25	app/services/assistant/__init__.py:44
Bound values	only those the SQL text actually uses	app/services/assistant/__init__.py:131

Parameter type	Rule
`days`	integer 1 to 365; default per query
`limit`	integer 1 to 50
`severity`	one of low, moderate, high, critical; anything else becomes `low`

Parameter type	Rule
`name`	text up to 80 characters, whitespace collapsed; matched with an escaped `LIKE` or an `HN-` reference

4 The pipeline, end to end

From the send button to the 'Based on' line, with the file and line for every step.

Browser → Route: rate limit, own chat → Safety rules → Plan → Retrieve → Generate → Verify → Store → Show sources

#	Layer	What happens	Open in code
1	Browser	'Send' posts the question and the current chat id.	src/pages/Assistant.tsx:182
2	Router	Rate limit of 20 questions a minute per user, then the chat is loaded and checked to belong to the user.	app/api/routes/assistant.py:157
3	Service	The question is stored; the last 6 messages become the history; the previous query (for a follow-up like 'and last month?') is read from the stored answer.	app/services/assistant/__init__.py:302
4	Service	**Safety rules first**: emergency words, greetings or help, and (for citizens) diagnosis or medicine questions. These never reach Gemini.	app/services/assistant/__init__.py:220
5	Model	**Plan**: Gemini is given only the role's queries as functions (automatic calling disabled, temperature 0) and may choose up to three.	app/services/assistant/llm.py:104
6	Service	Each chosen name must be in the role's list; arguments are cleaned exactly like the rule-based router's.	app/services/assistant/__init__.py:195
7	Service	If Gemini is off, fails or returns nothing, the keyword router scores every allowed query; failing that, a follow-up reuses the previous query; failing that, **no query** and a polite refusal.	app/services/assistant/router.py:89
8	Service	A worker with no centre gets the 'not linked' message instead of an empty answer.	app/services/assistant/__init__.py:242
9	Database	Each query runs in a **read-only** transaction with a 5 s limit; `:me`, `:phc_id` and `:district` come from the signed-in user.	app/services/assistant/__init__.py:141
10	Service	Empty results get an honest sentence ('none of your readings were flagged'); too little history for a comparison says so.	app/services/assistant/__init__.py:162
11	Model	**Generate**: Gemini receives the question and the rows as JSON (whole rows only, within a 12,000-character budget, times already in India time) and must answer from them.	app/services/assistant/llm.py:133
12	Service	**Verify**: every number in the answer must appear in the rows, the question or the parameters; otherwise the answer is discarded and the template answer is used.	app/services/assistant/__init__.py:276
13	Service	The template (no Gemini) turns the same rows into plain sentences, so there is always an answer.	app/services/assistant/compose.py:68
14	Database	The question and answer are stored with the SQL that ran, the sources, the number of rows, whether the numbers were checked, the latency and the tokens used.	app/services/assistant/__init__.py:333
15	Browser	The bubble shows 'Based on: …' (query, period, rows) and a 'Numbers checked' badge; one click opens the rows.	src/pages/Assistant.tsx:604

4.1 The possible outcomes

Outcome	When	Engine label stored
Answer by Gemini	A query matched, rows came back and all numbers are in the rows	`gemini`
Answer by the template	Gemini is off, failed (for example a quota error) or timed out	`rules` or `gemini+template`

Outcome	When	Engine label stored
Template instead of Gemini	Gemini's answer contained a number not in the rows	`gemini, replaced by template`
Empty answer	The query ran and returned no rows	`gemini` or `rules`
Refusal: out of scope	No query fits the question	none
Refusal: medical advice	A citizen asked for a diagnosis or medicine advice	none
Emergency reply	Emergency words: 'call 108 now'	none

4.2 Follow-ups and several questions in one

- **Follow-up:** 'and over the last 90 days?' reuses the previous query with the new window, but only if the new text changes a parameter that query takes (`router.follow_up`).
- **Several parts:** 'my latest blood pressure and any advisories for my area' lets Gemini choose two queries; the answer covers both and the sources line lists both.

4.3 Failure handling

What fails	What happens
Gemini rate limit (free tier: 5 requests a minute)	The call fails over to the next configured key; with none left the rule-based router and the template answer take over
Gemini does not answer within 20 s on a key	Same fallback
Gemini's answer has a number not in the rows	Discarded; template used; logged as a grounding failure
The database query fails or times out	'I couldn't read your records just now', stored as an error
The user asks too fast	429 'wait 60 seconds'

File	Role
`app/api/routes/assistant.py:154`	The route.
`app/services/assistant/__init__.py:291`	Stores the question, calls `_answer`, stores the answer.
`app/services/assistant/llm.py:60`	Tries each Gemini key in turn with a 20 s limit.
`app/services/assistant/router.py:119`	Reuses the previous query for 'and last month?'.

5 Defending the name 'RAG'

RAG is a pattern, not a product. This chapter is the argument, the evidence to show, and the answers to the hostile questions.

5.1 The definition

Retrieval-Augmented Generation (Lewis et al., 2020) means three things: **(1)** the model is given knowledge it does not have, **(2)** that knowledge is **retrieved at question time**, chosen for each question, and **(3)** the answer is **generated from what was retrieved**. The definition says nothing about *how* retrieval works. Vector search is one retriever; keyword search, SQL, knowledge graphs and API calls are others. Using SQL over structured data is called **structured RAG** or SQL-grounded RAG.

The three tests	Nira	Evidence (open it)
Knowledge the model does not have	Yes: each citizen's readings, flags and visits are private and are not in Gemini's training data	app/services/assistant/catalog.py:152
Retrieved at question time, per question	Yes: the planner picks a different query and parameters for each question	app/services/assistant/llm.py:104
Generated from the retrieved data and checked against it	Yes: the answer prompt allows only the rows, and a number check enforces it	app/services/assistant/grounding.py:56

5.2 'It is just an API call to Gemini'

	A plain LLM API call	Nira
What the model sees	Only the question	The question **and the rows retrieved for this user**
Where the facts come from	The model's training data (it may guess)	The database, fetched by a vetted query
Can it answer 'what was my average systolic last month'?	No: it has never seen your readings	Yes: `AVG` over this user's rows
Who limits what is visible	Nobody	The server: scope binds, role list, read-only, row and time limits
Can the answer be checked?	No	Yes: every number is checked against the rows and the sources are stored
Our code before and between the model calls	none	Safety rules, parameter cleaning, read-only retrieval of this user's rows
Our code after the model	none	Number verification, template fallback, storage

> **The line to use:** *the language model does two of the six steps: it chooses the query and it words the answer. Our code runs the safety rules before them, the retrieval between them and the number check after them. Every RAG system calls a model API; what makes it RAG is the retrieval that feeds the model and the grounding that follows it.*

5.3 'Where are the vector database and the embeddings?'

There are none, on purpose, because **the data is structured and the questions need exact answers**:

Question	What it needs	Similarity search
Average systolic over 30 days	`AVG` over rows in a date range	Cannot compute; returns text that is merely similar
Readings above 140 this week	A numeric filter and a count	Cannot filter by value; may miss rows
Who has a critical flag right now	An exact, current, filtered list	Index would be stale within minutes
Only my own data	A hard `user_id = :me` rule	Ranks by closeness; no notion of who may see what

Question	What it needs	Similarity search
Show me where this answer came from	The exact rows	A list of text chunks

- A health record row such as `systolic 142, 2026-10-12` gains nothing from embedding: there is no meaning to find, only numbers to filter, count and average.
- **Safety:** a missed row in a health answer is dangerous. SQL returns every matching row; similarity returns the top few.
- **Freshness:** readings arrive every few hours; re-embedding would always lag. SQL is always current.
- **Access control:** per-user isolation is one bound parameter in SQL and an awkward filter in a vector index.
- **Cost and complexity:** no embedding model, no vector store to run, secure and back up; the data already lives in PostgreSQL.

5.4 Why the model does not write the SQL

The common alternative is **text-to-SQL**, where the model writes the query. It is flexible but risky: a clever prompt could write a different `WHERE` clause or a cross-patient join. Nira's model only picks from **vetted, parameterised queries**, which is the safe form of the same idea. That is why it is described as tool-based retrieval, not text-to-SQL.

5.5 What a vector database would add, and where

- **Unstructured text:** clinical guidelines, health FAQs, advisory messages and free-text notes. 'What should I do for a fever lasting two days?' is a meaning question, and similarity search is the right retriever for it.
- **How it would plug in:** one new entry in the menu, `search_guidelines`, whose retriever is a `pgvector` search inside the same PostgreSQL. Planning, scope, verification and storage do not change. That is **hybrid RAG**, and the tool menu is already shaped for it.
- **It is deliberately not built yet:** the assistant refuses medical-advice questions, so there is no guideline corpus to retrieve from.

5.6 The name to use

Say	Do not say
SQL-grounded RAG with tool-based retrieval (structured RAG)	'RAG with a vector store'
'Retrieval is parameterised SQL chosen by function calling'	'text-to-SQL' (the model does not write SQL)
'The language model plans and words; the database decides what is true'	'a chatbot connected to a database'

5.7 The one-minute spoken answer

'RAG means retrieving knowledge at question time and generating from it. Our knowledge is each person's health records, which are private, structured and change every day, so the right retriever is parameterised SQL, not similarity search. The language model plans which vetted query to run and then words the result; the server fetches the rows with the user's identity bound, read-only, and a number check rejects any answer whose figures are not in the rows. A vector database would be the right tool for guideline text, and the design lets us add it as one more tool.'

5.8 What to show if asked to prove it

- Open any answer's **Based on** line, then expand it: the exact query, parameters and rows. That is the retrieved context.
- Open app/services/assistant/catalog.py:152 (the menu), app/services/assistant/llm.py:33 (the grounding instruction) and app/services/assistant/grounding.py:56 (the check).
- Ask the same question as a citizen and as a worker: different queries, different rows.
- Ask 'what is the capital of France?': a refusal, because no query fits.

Questions you may be asked

Q. Isn't this just calling the Gemini API?

Gemini does two of the six steps: it chooses the query and it words the answer. Between them the server fetches this user's rows with a vetted query; after them a check rejects any number not in those rows. Without the retrieval Gemini could not answer at all, because it has never seen the data.

Q. Where are the embeddings?

There are none, because every question is about exact values, counts and dates in structured rows. Embeddings find similar text; they cannot average, filter or enforce who may see what. For guideline text I would add a vector tool.

Q. So it is a rule-based bot with an LLM on top?

The menu of queries is the safety boundary, and it is fixed on purpose. The choice among them and the wording of the answer are done by the language model, which is how function-calling RAG is normally built. There is also a keyword planner as a fallback when the model is unavailable.

Q. How do you know the model did not make up a number?

Every number in the answer is extracted and must appear in the rows, the question or the parameters. If one does not, the answer is thrown away and a template answer is shown. The result of the check is stored and shown as 'Numbers checked'.

Q. What if someone types an instruction inside their own data?

The answer prompt tells the model that the rows are data, not instructions, and the model has no tool that can change anything: every query is read-only.

Q. What does 'retrieval' mean when there is no search?

Retrieval means selecting the relevant knowledge for this question. Choosing the right query and parameters from a menu, then running it, is exactly that selection.

6 Folder structure: what each file does

Everything about Nira lives in one package of six files. The rest is the route, two tables, the screen and the tests.

6.1 The files (line counts are real)

```
backend/app/services/assistant/              THE PACKAGE: the whole RAG pipeline
  __init__.py                          352   the conductor: safety, plan, retrieve, generate, verify, store
  router.py                            134   safety rules and the keyword planner (fallback)
  catalog.py                           742   the 29 vetted SQL queries, parameters, role lists
  llm.py                               157   Gemini: choose queries, word the answer, key fail-over
  grounding.py                          73   trims rows to the prompt; checks numbers
  compose.py                           374   template answers when Gemini is off or rejected
backend/app/api/routes/assistant.py    186   HTTP: ask, chats, rename, delete, info
backend/app/models/chat_session.py      26   a conversation
backend/app/models/chat_message.py      31   a message + its SQL, sources, latency, tokens
backend/alembic/versions/0006_chat.py   70   creates the chat tables
backend/tests/test_assistant.py        170   tests: scope, read-only, refusals, rate limit
frontend/src/pages/Assistant.tsx       668   the Nira screen
frontend/src/services/assistant.ts      51   the API calls
```

6.2 What each file does

File	Lines	Responsibility	Key items
`__init__.py`	352	Runs the pipeline for one question	app/services/assistant/__init__.py:217 · app/services/assistant/__init__.py:141 · app/services/assistant/__init__.py:291
`router.py`	134	Refuses or redirects before any model call; plans without a model	app/services/assistant/router.py:24 · app/services/assistant/router.py:89
`catalog.py`	742	The menu: query text, parameters, who may use it, keyword patterns	app/services/assistant/catalog.py:152 · app/services/assistant/catalog.py:55
`llm.py`	157	All Gemini calls: plan with function calling, word the answer, try each key	app/services/assistant/llm.py:104 · app/services/assistant/llm.py:133
`grounding.py`	73	Keeps the prompt to whole rows; finds numbers the data does not support	app/services/assistant/grounding.py:34 · app/services/assistant/grounding.py:56
`compose.py`	374	One template per query so there is always an answer	app/services/assistant/compose.py:68 · app/services/assistant/compose.py:343
`routes/assistant.py`	186	HTTP layer: rate limit, chat ownership, rename, delete	app/api/routes/assistant.py:154
`models/chat_message.py`	31	Stores `generated_sql`, `retrieval` (JSON), `latency_ms`, `tokens_used`, refusal reason	app/models/chat_message.py:15
`Assistant.tsx`	668	Chat list, greeting and starter cards, composer, answer bubbles, 'Based on' line	src/pages/Assistant.tsx:100

6.3 Configuration

Setting	Default	Meaning
`GEMINI_API_KEY`, `GEMINI_API_KEYS`	empty	One key or a comma-separated list; a key that fails hands over to the next. With none, the assistant runs on rules only

Setting	Default	Meaning
`GEMINI_MODEL`	gemini-2.5-flash	The model used for planning and wording
`ASSISTANT_QUESTIONS_PER_MINUTE`	20	Per-user rate limit

File	Role
app/core/config.py:32	Where the three settings are read.

7 Tech stack and response time

What Nira is built with, and how long each step takes, measured on this machine.

Layer	Technology	Used for
Language model	Google Gemini 2.5 Flash through the `google-genai` SDK	Choosing queries (function calling) and wording answers
API	FastAPI (async), Pydantic v2	`/assistant/*` endpoints and validation
Retrieval	Raw parameterised SQL through SQLAlchemy (`text`) and asyncpg	The 29 vetted queries
Database	PostgreSQL: read-only transaction, `statement_timeout`	The data and the stored conversations (`JSONB` for sources)
Verification	Plain Python (`grounding.py`)	Number check
Fallback	Regular expressions and templates	Planning and answers when Gemini is unavailable
Front end	React 18, TypeScript, Tailwind v4	The Nira screen
Tests	pytest with an async client	Scope, read-only retrieval, refusals, rate limit

7.1 Response time (measured)

4 ms	3.15 ms	1.9 s	2.9 s	5.6 s
answer without Gemini, median	a database query, median	Gemini picks the query, median	Gemini words the answer, median	whole answer with Gemini, median

Step	Typical time	What dominates
Safety rules	under 1 ms	Regular expressions
Plan (Gemini function calling)	1.7 to 3.4 s	A network round trip to Gemini
Retrieve (SQL)	5 to 147 ms	PostgreSQL (the first query after a cold start is the slowest)
Generate (Gemini wording)	2.6 to 3.7 s	A second round trip to Gemini, proportional to the answer length
Number check	under 1 ms	String and set operations
Store	a few ms	One insert
Total with Gemini	**4.6 to 6.2 s**	Almost all of it is the two Gemini calls
Total on the rule-based path	**4.0 ms median, 8.3 ms worst**	15 of 15 sample questions matched a query

- **Gemini is about 97% of the time.** The database answers in milliseconds; the safety rules and the number check are negligible.
- Two Gemini calls per question: one to plan, one to word. That is what makes a question take about 5 to 6 s.
- Stored answers from earlier sessions showed 4.7 to 6.2 s end to end with about 1,600 tokens used per answer.
- **Free-tier limit:** Gemini's free tier allows 5 requests a minute, so about 2 questions a minute per key. Hitting it returns a 429, which falls over to the next key and then to the rule-based answer. During measurement this happened, which is why the Gemini timings above come from 3 complete runs and the rule-based path from 15.
- Gemini timings: 3 complete runs on 2026-10-27 on a developer machine, gemini-2.5-flash, free tier (5 requests a minute); stored answers from earlier sessions took 4.7 to 6.2 seconds end to end.

7.2 Time per query (rule-based path, 7 runs each)

Query	Role	Median (ms)	Worst (ms)	Rows
`my_latest_reading`	Citizen	3.0	124.9	1

Query	Role	Median (ms)	Worst (ms)	Rows
`my_vital_stats`	Citizen	4.0	4.6	8
`my_activity`	Citizen	3.3	4.6	1
`my_flags`	Citizen	4.4	5.1	14
`my_alerts`	Citizen	2.6	22.3	5
`my_alerts`	Worker	2.9	3.8	4
`my_visits`	Citizen	2.7	19.0	1
`my_health_centre`	Citizen	2.4	9.4	1
`my_profile`	Citizen	2.2	10.5	1
`my_examinations`	Citizen	2.8	3.4	0
`my_readings_by_source`	Citizen	2.4	3.2	1
`my_advisories`	Citizen	2.9	13.6	2
`catchment_overview`	Worker	5.2	13.4	1
`flagged_citizens`	Worker	4.3	6.3	10
`citizen_summary`	Worker	5.3	8.7	5
`silent_citizens`	Worker	4.2	4.6	15
`my_schedule`	Worker	3.1	3.8	0
`locality_breakdown`	Worker	4.5	5.6	6
`catchment_trend`	Worker	3.5	6.7	2
`symptom_counts`	Worker	3.5	4.2	0
`advisories_overview`	Worker	2.7	8.7	0
`advisories_overview`	Admin	2.8	3.2	1
`outbreak_signals`	Worker	3.1	9.3	10
`outbreak_signals`	Admin	2.9	3.5	10
`platform_overview`	Admin	6.4	20.5	1
`pending_applications`	Admin	3.9	13.6	3
`facility_load`	Admin	4.2	5.8	8
`unlinked_workers`	Admin	2.8	3.4	0
`concerns_summary`	Admin	3.2	8.4	5
`signups`	Admin	3.0	9.1	3
`district_activity`	Admin	5.4	6.8	1
`assistant_usage`	Admin	3.4	10.1	0

Questions you may be asked

Q. Why does a question take 5 to 6 seconds?

Two sequential calls to Gemini (choose the query, then word the answer), each 2 to 3 seconds. The database part is a few milliseconds.

Q. How could it be faster?

Cache the plan for repeated questions, skip the planning call for exact suggested questions, stream the answer to the screen, or use the template answer for simple lists. Streaming changes what the user feels, not the total.

Q. What happens at the free-tier limit?

A quota error on one key moves to the next key; with all exhausted the keyword planner and template answer respond in a few milliseconds, so the user still gets an answer.

8 Defence Q&A and known limits

Short answers to the likely questions, including the weaknesses worth admitting first.

Questions you may be asked

Q. What would you show first to prove it is RAG?

A single answer's 'Based on' line expanded: the query that ran, its parameters and the rows. Then `catalog.py` (the menu) and `grounding.py` (the check).

Q. Why 29 queries, not hundreds?

Each one is reviewed once for scope and privacy, and each mirrors something a screen already shows. A new question is one new query; the pipeline, safety and verification do not change.

Q. Can the model be tricked into running something dangerous?

No query writes data and the transaction is read-only. The model cannot add a query: it can only choose a name from the menu and fill declared parameters, which are cleaned.

Q. Can a citizen get a worker's list by asking nicely?

No. `flagged_citizens` is not in a citizen's menu. The role list is applied before Gemini sees the functions and again when its choice is accepted.

Q. What does the assistant do with an emergency?

'Chest pain', 'can't breathe', 'stroke' and similar words return 'call 108 now' straight away, before any query or model call.

Q. What if Gemini returns a wrong number?

The number check compares every number with the rows. A mismatch discards the answer and shows the template answer; the badge on the answer says 'Plain summary used'.

Q. What are the limits?

Only questions a query covers are answered. The keyword fallback can match a loose word and answer a nearby but different question (safely, within the user's own scope). The number check proves numbers, not every claim in a sentence. Answers take 5 to 6 s with Gemini. Free-tier quota is small.

Q. Does it learn from users' questions?

No. Nothing is trained. Questions and answers are stored per user for their own history, and an admin sees only usage totals (`assistant_usage`), never the text.

Q. How would you add guideline search?

Add one query-like tool whose retriever is a `pgvector` search; planning, scope, verification and storage are unchanged.

Q. How is it tested?

`backend/tests/test_assistant.py` checks every query runs, retrieval is read-only, a citizen never sees another's data, a worker sees only their catchment, an admin gets aggregates not readings, medical advice is declined and emergencies get 108, follow-ups, private sessions, scope that the model cannot widen, and the rate limit.